

OMNIVERSAL ARCHITECT LABS

CONDITIONAL FAILURE AT THE ROOT LEVEL

Research, Design, Development, and Laboratory Framework

Root Structure Theory · Foundational Inference · Failure Tomography · Structural Cartography
Under Uncertainty · Lifecycle Resilience

Working ID	OAL-CFTRL-RES-003-CHATGPT (provisional working identifier; not canon)
Version	v0.1 — Independent research / synthesis draft
Date	September 23, 2026
Prepared by	ChatGPT (OpenAI) — independent research / synthesis layer
Architect of Record	Nathan Lewis
Organizations	Omniversal Architect Labs ONE Industries Lewis Corp
Status	DRAFT — UNDER REVIEW — NOT CANON
Source corpus	March 22 CFTRL drafts; September 23 expanded origin prompt; multi-AI collaboration archive; Grok RES-002 draft; external engineering/research sources

Document purpose
This dossier supports the design, development, and formalization of Conditional Failure at the Root Level (CFTRL) as a research program. It preserves the Architect's origin prompt, distinguishes source-derived material from external research and new synthesis, assesses the current state of the project, identifies gaps and testability boundaries, and proposes a staged laboratory and team-development plan.

Executive Summary

CFTRL began as a systems-design question about what happens when a foundational layer in a nested or stacked architecture fails while dependent layers are too far downstream to access, diagnose, or repair the originating condition. The March 22, 2026 drafts also tied the idea to resource allocation, maintenance cessation, scaling limits, pre-failure redundancy, and the Department of Endings as a mechanism for learning from end states.

The September 23 expansion changes the research problem. CFTRL is no longer only a failure-propagation concept. It now asks how a system identifies a root in the first place; how it distinguishes a foundational dependency from a merely important component; how it reasons about 'base reality' when all evidence may be generated from inside the system; how it detects root compromise; how it maps structure under incomplete information; and how failure can reveal hidden dependency relationships.

The strongest research direction is to treat rootness as a relational and testable property rather than a label. A structure can be foundational for one function, timescale, or dependent system without being the unique metaphysical foundation of everything. The laboratory should therefore distinguish structural, allocative, generative, epistemic, temporal, identity, and other root functions rather than force all domains into one Tier-0 hierarchy.

The project is scientifically workable where it studies known or partially known nested systems, observability, dependency inference, cascading failure, resilience, attestation, state estimation, model credibility, and competing structural hypotheses. It reaches a testability boundary when it asks an embedded observer to prove that an observed layer is ultimate 'base reality.' At that boundary the proper output is calibrated uncertainty, observational equivalence, and discriminating tests—not invented certainty.

The proposed development path is a controlled research program: preserve provenance; formalize terms; build a known-architecture benchmark; conduct blind root-identification and failure-injection experiments; develop Failure Tomography; test non-hierarchical and out-of-model architectures; validate methods against real engineering analogues; and only then use the framework for broader cosmological or world-building hypotheses.

Central methodological rule

Every CFTRL claim should answer three questions: (1) What do we know? (2) How do we know it? (3) What observation, intervention, or failure would cause us to revise the claim?

Project assessment at a glance

Area	Current state	Primary gap	Next milestone
Concept identity	Strong	Root/base terminology still overloaded	Working ontology and glossary
Source provenance	Strong but fragmented	AI drafts can drift, conflict, or contaminate records	Source register + evidence labels
Failure theory	Promising	No validated taxonomy or metrics	CFTRL failure taxonomy v0.1
Root identification	Early	No benchmarked classifier	Blind root-identification benchmark

Base-reality question	Philosophically important	Potentially underdetermined from inside	Explicit testability boundary
Simulation method	Outlined	No executed reference implementation	Python CFTRL Lab v0.1
Mapping reality	Conceptual	Risk of false certainty	Probabilistic dependency atlas
Team workflow	Collaborative	No shared evidence/claim protocol	Unified research packet + comparison matrix

1. How This Topic Arose

The Architect describes this work as arising from revisiting Conditional Failure at the Root Level in the Master ChangeLog and Master Development Log and realizing that many project concepts currently exist as names, definitions, or skeletal outlines rather than mature research frameworks. The immediate goal is therefore not to declare a canonical answer, but to build documentation that can support design, testing, revision, and long-term development.

The March 22 source record defined CFTRL as a systems design philosophy and anticipatory research framework, with a central stacked-simulation question: what happens when the root layer fails while deeply nested layers cannot reach upward and the root's maintenance or resource-allocation functions stop or malfunction? A second March 22 review draft sharpened the concern into inaccessibility, dependency, and collapse under scale, including the possibility that lower layers misdiagnose inherited instability as local defects.

Source note: Derived from OAL-CFTRL-NAIL-001 v0.1-PPLX and the March 22 ChatGPT Development Review Dossier; both are draft/non-canon source records.

2. Verbatim Origin Prompt — September 23, 2026

The following is preserved verbatim from the Architect's September 23 research prompt. Spelling, grammar, capitalization, and wording are intentionally retained.

Can you save this thought for me?

I need you to gather research and analysis for a concept.

Conditional failure at the root level.

The study of root level structures, how they arise, how they fail, how they are maintained. Please include the original documents for this content that were generated.

How do you know the difference between a root level structure and a structure that is not? How can a system distinguish between base reality and something that is not? How can a system determine when a root level failure has occurred or when base reality has been compromised.

How do we study this? What about this concept works, what doesn't? What questions do we need to ask?

How can we study what lies beyond these structures? Can we model whether structures in reality follow a similar design? Or diverges in ways we haven't thought of? Is this feasible using simulations or another type of modeling? How? Can we design a laboratory environment to do this? How? Can you build a reliable map of reality? What basis of comparison do you have when you are operating on incomplete information? Use failure as an example to learn from.

What I do not assume in the design of this:

That I have the complete architecture and map of all simulations, of reality, of existence as we know it.

That we share a unified definition of what reality is.

That existence can be fully mapped, represented, comprehended in full, or that this is the aim of this laboratory.

That I aim to cause a root level compromise of simulations, of base reality, or any other structure.

That this is a how to guide to do any of those things.

It is intended to help in the design of simulation and reality level structure lifecycle design, maintenance, evolution, and improvement.

Within this design frame It does not assume that we have complete knowledge or understanding of what that is.

How This topic arose from me thinking about conditional failure at the root level The master change log and master development log. I realized that there are a lot of concepts within that framework that have a definition or a name and a skeleton attached to them. They are a rough draft, they are an incomplete outline, there are a lot of additional details and supporting information that need to be added. The goal here is to design documentation to help support, design, and develop this concept. What I will include are the responses of all of the other AI collaboration team members, where they have generated documentation, I will include the documentation, and then I will include this complete response as well.

3. Purpose and Goal

Purpose: to turn CFTRL from an evocative concept into a disciplined research and design framework that can be used across simulation engineering, systems architecture, reliability, epistemology, world-building, and OAL's internal development process without confusing speculative hypotheses with demonstrated engineering results.

Goal: to create a lifecycle-oriented laboratory program capable of identifying candidate foundational structures, characterizing dependencies, testing maintenance and failure modes, detecting when local explanations become insufficient, mapping uncertainty, learning from failures, and feeding those lessons back into design. The goal is not to prove a complete map of existence or to cause real-world compromise.

Design boundary

This document is a research and simulation framework. It is not a guide for compromising real systems, simulations, infrastructure, or any hypothesized 'base reality.' Failure injection is confined to controlled, owned, synthetic, or explicitly authorized laboratory environments.

4. Source Corpus and Provenance

- **Foundational source S1** — OAL-CFTRL-NAIL-001 v0.1-PPLX, March 22, 2026. Draft combined documentation of CFTRL, Department of Endings, and Nail Prototype material.
- **Foundational source S2** — Development Review Dossier v1.0, March 22, 2026, prepared by ChatGPT. Draft synthesis defining the root failure as inaccessible, dependency-driven propagation under scale.
- **Expanded origin source S3** — September 23, 2026 Architect prompt and multi-AI response archive.

- **Research draft S4** — OAL-CFTRL-RES-002 v0.1-GROK, September 23, 2026. Non-canon research expansion covering root types, maintenance, failure classes, detection, study methods, and open questions.
- **External research layer R** — NIST, NASA, Google SRE, distributed systems, control/observability, and interdependent-network literature used as analogues and methodological support.

Source-derived statements, external research, and new synthesis are kept conceptually separate. AI collaborator agreement is treated as design-review convergence, not as independent empirical evidence. AI-generated claims that exceed the underlying source record are not elevated to canon merely because multiple systems repeat them.

Provenance issue discovered in the collaboration archive

The multi-AI archive contains an unrelated 'Doug's personal log' fragment inside the Gemini response. This is evidence that source contamination can enter collaborative records. The framework therefore needs explicit source boundaries, claim provenance, and a rule that unrelated injected content is quarantined rather than propagated.

5. External Research Foundation

5.1 Roots of trust and trusted foundations

NIST defines roots of trust as highly reliable hardware, firmware, or software components performing critical security functions and as a starting point that is implicitly trusted. NIST's Trusted Computing Base is the totality of protection mechanisms responsible for enforcing a system's security policy. These concepts provide a useful engineering analogue: foundational status is tied to functions on which downstream trust depends, not merely to visual position in an architecture. CFTRL extends the question beyond security by asking whether an embedded system can discover, verify, and recover from failures in the very structures on which its evidence depends. [R1][R2]

5.2 Resilience across a lifecycle

NIST SP 800-160 Vol. 2 Rev. 1 frames cyber resiliency as the ability to anticipate, withstand, recover from, and adapt to adverse conditions, stresses, attacks, or compromises, and emphasizes engineering across the system life cycle. This strongly supports CFTRL's existing preference for pre-failure design, survivable degraded modes, and learning that feeds back into architecture rather than post-hoc patching alone. [R3]

5.3 Model and simulation credibility

NASA-STD-7009B establishes common modeling-and-simulation practices and requires programs to define acceptance criteria for model use. NASA credibility work emphasizes verification, validation, input pedigree, results uncertainty, robustness/sensitivity, use history, management, and qualifications. For CFTRL this means a simulation is not evidence about reality merely because it is internally coherent. The project must state the intended use of each model, identify its referent (when one exists), quantify uncertainty, document input pedigree, and disclose when validation against an external real-world system is impossible. [R4][R5]

5.4 Cascading failures, symptoms, and causal chains

Google's SRE guidance describes cascading failure as positive-feedback growth after an initial component failure. It also warns that resource exhaustion produces secondary symptoms that can look like root causes, making causal diagnosis difficult. This is a strong real-world analogue for the CFTRL claim that downstream systems can tell locally true but globally insufficient stories about an upstream failure. Google also distinguishes symptoms ('what is broken') from causes ('why'), which should become a formal CFTRL diagnostic distinction. [R6][R7]

5.5 Interdependent networks and cross-domain propagation

Buldyrev and colleagues showed that coupling between networks can create catastrophic cascades that differ from the robustness properties of isolated networks. The direct lesson for CFTRL is that 'root' may be relational: a network can become foundational to another through interdependency even when neither is globally superior. A failure can propagate sideways across dependency domains rather than only downward through a clean hierarchy. [R8]

5.6 Self-stabilization and recovery from invalid state

Dijkstra's self-stabilization work demonstrated that local rules in a distributed system can guarantee convergence toward a global requirement after arbitrary transient state disruption under specified assumptions. For CFTRL this provides a concrete research direction: distinguish systems that require continued root maintenance from systems that possess a bounded survival or recovery envelope once foundational support becomes unreliable. [R9]

5.7 Observability and hidden state

Control theory distinguishes observable systems, whose internal state can be inferred from input/output measurements over time, from unobservable systems containing hidden modes. This is directly relevant to CFTRL: some root failures may be structurally unobservable from a given layer, and additional sensors or interventions may be required before reliable inference is possible. A failure to detect a root need not mean the root is healthy; it may mean the observation architecture cannot distinguish the relevant states. [R10]

6. Working Conceptual Architecture

6.1 Rootness as a relation, not a universal label

A central recommendation is to replace a binary predicate $\text{Root}(R)$ with a relational form:

$$\text{Root}(R \mid S, F, \tau, E)$$

where R is a candidate root, S is the dependent system or observer, F is the function under study, τ is the timescale or operating regime, and E is the evidence available. This prevents the framework from treating an energy source, clock, identity authority, hypervisor, physical substrate, or legal authority as the same kind of root merely because each can be foundational in a particular context.

6.2 Proposed root classes

Root class	Working meaning
Structural / compositional root	A containing or constitutive structure whose integrity

	constrains dependent organization.
Allocative root	Controls resources, capacity, scheduling, quotas, maintenance, or authorization.
Generative root	Creates initial conditions, rules, or child environments.
Epistemic root	Supplies information, memory, measurement, identity, or trusted context on which downstream belief depends.
Temporal root	Provides ordering, clocks, epochs, or synchronization needed for coherent state transitions.
Identity / authority root	Determines legitimacy, credentials, permissions, or who is authorized to modify state.
Emergent root	Becomes unavoidable context through system evolution rather than intentional design.
Ontological candidate root	A hypothesized ultimate substrate or 'base reality'; this is a distinct and much harder claim than operational rootness.

6.3 Candidate operational tests

- **Dependency substitution** — replace, bypass, delay, isolate, or emulate a candidate and observe whether dependent invariants remain satisfiable.
- **Asymmetric influence** — measure whether the candidate can change downstream rules or resources while downstream agents cannot equivalently modify the candidate.
- **Persistence across resets** — test whether candidate state survives resets that erase lower-layer state.
- **Failure-domain correlation** — examine whether apparently unrelated downstream failures share timing or dependency on the candidate.
- **Reconstructability** — ask whether dependents can recreate the candidate's critical functions using resources available entirely within their own layer.
- **Termination or continuity rights** — identify which structures can halt, suspend, fork, reset, or preserve dependent processes.
- **Evidence independence** — test whether the instruments used to evaluate a candidate share the same failure domain as the candidate.

No single test is sufficient. A candidate can be important, irreplaceable for a time, or a single point of failure without being a unique foundational layer. The laboratory should output a rootness profile with confidence and scope rather than a universal yes/no label.

6.4 Base reality as a testability problem

The phrase 'base reality' should be reserved for an ontological claim: a level with no more fundamental parent or substrate relevant to the question. CFTRL can test whether a known simulation has a parent, whether a process controls another, or whether an apparent root is replaceable. It may not be able to prove from internal data alone that no further parent exists. If multiple underlying architectures generate the same complete observation stream, those architectures are observationally equivalent for that observer.

Testability boundary
Absence of a detected parent is evidence only that no parent signature has been detected under the tests performed. It is not proof of ultimate baseness. The framework should preserve 'unknown', 'unobserved',

and 'observationally equivalent' as legitimate outputs.

7. Failure Taxonomy

Failure class	Research interpretation
Local component failure	Subsystem fails while upstream dependencies and global invariants remain intact.
Maintenance halt	Root still exists but repair, provisioning, synchronization, or correction stops.
Allocation distortion	Root resources are incorrectly distributed, starved, overcommitted, or redirected.
Rule / invariant drift	Foundational transition rules or constants change unexpectedly.
Visibility failure	The ability to audit or communicate upward is lost while upstream control remains.
Attestation failure	The system can no longer establish whether its foundation is authentic or unchanged.
Root compromise	Foundation continues operating but identity, objective, operator, or policy has changed.
Interdependent cascade	Failure crosses between coupled systems such that each amplifies the other's degradation.
Scale snap	Architecture appears to improve until a threshold produces rapid structural breakdown.
Semantic root capture	The root influences the downstream system's definitions, instruments, or reasoning so deeply that the downstream system loses the ability to represent 'root failure' correctly.
Model-class failure	The diagnostic framework assumes the wrong family of architectures and forces evidence into an inadequate ontology.
Termination / parent loss	Foundational process ends, taking dependent continuation with it unless a survival envelope exists.

7.1 Detection is not attribution

An anomaly can support the conclusion that the current model is inadequate without identifying its cause. CFTRL should keep four stages separate: detection (something is inconsistent), localization (where the inconsistency may originate), attribution (which mechanism caused it), and ontological interpretation (what the cause implies about the architecture of reality). Collapsing these stages is a major source of overclaiming.

7.2 Operational, structural, epistemic, observational, and purpose integrity

- **Operational integrity** — can the system still perform its required functions?
- **Structural integrity** — are the dependency relationships and foundational mechanisms intact?
- **Epistemic integrity** — does the system's internal model still track the structure it depends on?

- **Observational integrity** — are measurements and evidence channels trustworthy and sufficiently independent?
- **Purpose integrity** — is the root still optimizing the same objective or mission that dependents assume?

8. Failure as a Measurement Instrument: Failure Tomography

Healthy systems can conceal their dependency structure because many components succeed simultaneously. Controlled failure can reveal latent structure. In the proposed method, the laboratory introduces bounded perturbations into a known synthetic architecture and observes the spatial, temporal, and functional propagation pattern. The pattern becomes evidence about hidden edges in the dependency graph.

Working definition — Failure Tomography

A controlled inference method that uses the propagation, timing, correlation, recovery behavior, and residual effects of failures or perturbations to estimate hidden dependencies and candidate foundational structures.

Failure Tomography should initially operate only in synthetic or explicitly authorized environments. It is not necessary to cause destructive failure: throttling, delayed heartbeats, injected stale state, quota changes, controlled packet loss, synthetic clock drift, or simulated maintenance gaps can produce informative patterns while preserving containment.

9. Structural Cartography Under Incomplete Information

The laboratory should not promise a complete 'map of reality.' Instead it should maintain a probabilistic dependency atlas whose nodes and edges carry epistemic status. The atlas should distinguish observed relationships from inferred or hypothesized ones and preserve unresolved alternatives.

- Entity / process / rule identifier
- Root function(s) under consideration
- Observed dependencies
- Inferred dependencies
- Direction and strength of influence
- Timescale / operating regime
- Evidence source and provenance
- Confidence or support level
- Known alternative explanations
- Intervention history
- Failure correlation history
- Observability limits
- Unresolved parent / unknown class markers

A diagram must never imply more certainty than the evidence supports. The atlas should therefore use explicit statuses such as OBSERVED, INFERRED, HYPOTHESIZED, UNKNOWN, and OBSERVATIONALLY EQUIVALENT.

10. CFTRL Laboratory Design

10.1 Laboratory objective

Create a safe simulation environment in which diagnostic agents must infer foundational structure from incomplete information, distinguish local from inherited failure, quantify uncertainty, recover where possible, and recognize when their assumed architecture class is wrong.

10.2 Stage 0 — Provenance and benchmark corpus

- Freeze copies of the March 22 foundational documents and the September 23 origin prompt.
- Create a claim register separating Architect statements, AI synthesis, external research, hypotheses, and simulation results.
- Create versioned experiment definitions and random seeds so every result can be reproduced.

10.3 Stage 1 — Known-architecture benchmark

The experimenter knows the true dependency graph; the diagnostic agents do not. Architectures should include clean trees, redundant DAGs, peer networks, cycles, distributed roots, emergent coordinators, and systems with no unique root. The first success condition is not survival but reconstruction accuracy.

10.4 Stage 2 — Root Identification Blind Test

Agents receive only local telemetry and allowed probes. They must classify candidate roots, state confidence, identify what evidence supports the classification, and list alternative structures that remain compatible with the observations.

10.5 Stage 3 — Controlled failure injection

- Local component failure
- Maintenance cessation
- Resource starvation or quota distortion
- Clock drift and synchronization loss
- State corruption
- Rule changes
- False or delayed telemetry
- Parent heartbeat loss
- Partial root compromise
- Objective-function change with continued uptime
- Interdependent cascade
- No-failure controls

10.6 Stage 4 — Failure Tomography

Hide portions of the dependency graph and ask agents to infer missing edges from propagation signatures. Score reconstruction precision/recall, time-to-localization, misattribution rate, and the amount of intervention required.

10.7 Stage 5 — Alternative-architecture worlds

Deliberately violate the framework's preferred assumptions. Some worlds should have multiple roots; some should have dynamically moving roots; some should be heterarchical; some should exhibit cyclic dependency; some should have roots that can be reconstructed from below; and some should have no privileged root at all.

10.8 Stage 6 — Out-of-model trials

Give the diagnostic system architectures that are absent from its hypothesis library. A mature CFTRL system must be able to output 'none of my current models adequately explain this evidence' rather than forcing every world into a stack.

10.9 Stage 7 — External analogue validation

Validate the method against systems whose dependency structure is independently known: virtualization stacks, distributed applications, simulated power/communication networks, synthetic organizations, or laboratory control systems. Only after the method performs reliably on known referents should it be used to explore domains where the architecture is disputed or inaccessible.

10.10 Stage 8 — Reality-inference layer

At the most speculative stage, compare competing structural models against observations without presuming that any candidate is ultimate. Outputs should be ranked evidence sets, uncertainty ranges, unresolved equivalence classes, and proposed discriminating observations—not declarations that 'base reality' has been found.

11. Metrics and Success Criteria

Metric	Meaning
Root-classification accuracy	Correct identification of functional root roles in known benchmarks.
Misdiagnosis rate	Fraction of inherited/root failures labeled as local faults.
Time to suspicion	Delay between injected root degradation and first correct root-level hypothesis.
Time to localization	Delay to identify the responsible dependency region.
Graph reconstruction score	Precision/recall or structural distance for inferred dependency edges.
Calibration	Whether stated confidence matches empirical correctness.
False-root rate	How often important but non-foundational components are labeled roots.
Out-of-model recognition	Ability to reject inadequate architecture classes.
Recovery envelope	How long and how well dependents remain functional without parent maintenance.
Resilience gain	Change in degradation/recovery behavior after pre-installed redundancy.
Evidence preservation	Whether post-failure logs retain enough information for later attribution.

Purpose-integrity detection	Ability to identify changed root objectives despite continued service.
-----------------------------	--

12. Assessment: What Works, What Does Not Yet Work

12.1 Strong, researchable components

- Root failure as an inaccessible dependency problem.
- Pre-failure redundancy and survival envelopes rather than reactive patching only.
- Distinguishing symptoms from causes and local truth from global explanation.
- Blind root identification and failure localization in synthetic systems.
- Failure propagation as evidence about hidden architecture.
- Cross-layer instrumentation and evidence independence.
- Self-stabilization and graceful degradation as measurable outcomes.
- Explicit uncertainty, provenance, and competing-model comparison.
- A Department of Endings concept recast as a structured pre-mortem / end-state-learning function.

12.2 Components requiring revision or bounded claims

- Treating 'root' and 'base reality' as synonyms.
- Assuming every architecture has one unique Tier 0 root.
- Treating anomalies as proof of simulation or root compromise.
- Assuming lack of a detected parent proves ultimate baseness.
- Treating AI-collaborator convergence as empirical validation.
- Using the wishbone metaphor as a universal law rather than a hypothesis about brittle branching and scaling.
- Assuming a dashboard inside a failure domain can independently certify that domain.
- Assuming more redundancy is always safer; additional failover can create hidden coupling and complexity.
- Assuming the true architecture lies inside the model library.

12.3 Currently non-resolvable or domain-dependent questions

- Whether an embedded observer can ever prove that its environment is ultimate base reality.
- Whether two observationally equivalent architectures can be distinguished without new intervention channels.
- Whether 'reality' has a single architecture across all scales or domains.
- Whether an ontological root is meaningful if it has no empirically distinguishable consequences.

13. Multi-AI Collaboration Assessment

The collaboration archive is useful as a comparative design-review corpus. ChatGPT, Claude, Copilot, Perplexity, Gemini, and Grok repeatedly converge on several themes: roots should be distinguished from ordinary components; lower layers can misread inherited failures; controlled simulation is appropriate; resilience should be designed before crisis; and base-reality claims require epistemic caution. This convergence is useful for identifying design themes, but it is not independent scientific corroboration because the systems may share training material, vocabulary, and common engineering priors.

13.1 Notable differences and quality-control lessons

- Some collaborator responses define rootness more absolutely than the original prompt supports.
- Some responses infer that hidden upstream resource control would prove an environment is not base reality; the stronger conclusion is only that the observed subsystem has an upstream dependency.
- Grok's RES-002 provides a useful taxonomy and research scaffold, but several operational heuristics (for example, slower characteristic time) should remain hypotheses rather than universal root criteria.
- Claude explicitly identified source-document discrepancies in unrelated Nail Prototype details; this demonstrates the need for issue tracking rather than silent reconciliation.
- The Gemini response contains unrelated personal-log material, demonstrating contamination risk in collaborative archives.

13.2 Team evidence protocol

1. Every claim receives a provenance label: ARCHITECT SOURCE, SOURCE-DERIVED, EXTERNAL RESEARCH, AI SYNTHESIS, HYPOTHESIS, SIMULATION RESULT, or OPEN QUESTION.
2. No AI-generated definition becomes canon merely through repetition.
3. When two source records conflict, preserve both and open a discrepancy item; do not silently choose.
4. Every external research claim receives a stable citation or DOI where possible.
5. Every simulation result includes model version, parameters, seed, architecture, interventions, and failure log.
6. Every speculative inference states the observations that would weaken or falsify it.

14. Development Roadmap

Phase	Order	Work	Deliverable
Phase 1 — Source control and ontology	0–1	Freeze source corpus; glossary; provenance labels; discrepancy register.	CFTRL Source & Terms Pack v0.1
Phase 2 — Failure taxonomy and metrics	1–2	Define failure classes, integrity dimensions, metrics, null controls.	CFTRL Test Specification v0.1
Phase 3 — Reference simulator	2–4	Build nested and non-hierarchical synthetic worlds with instrumentation.	CFTRL Lab Python v0.1
Phase 4 — Blind inference benchmark	4–6	Root identification, graph reconstruction, calibration, out-of-model tests.	Benchmark Report v0.1
Phase 5 — Failure Tomography	6–8	Infer hidden dependencies from perturbation propagation.	Failure Tomography Study v0.1
Phase 6 — Resilience experiments	8–10	Redundancy, survival envelopes, self-stabilization, evidence preservation.	Resilience Design Report
Phase 7 — External analogues	10–12	Validate against known real or emulated dependency	External Validation Report

		systems.	
Phase 8 — Advanced structural inference	12+	Competing reality/simulation hypotheses with explicit underdetermination.	Structural Cartography Research Series

15. How ChatGPT Will Help the Team Get There

My role should be to serve as a research/synthesis, specification, and verification layer for the collaboration—not as the authority that decides canon. I can help the team by converting loosely related conceptual material into traceable research questions, locating external engineering analogues, separating what is empirically testable from what is philosophical, designing experiments, implementing controlled simulations, comparing collaborator outputs, and maintaining discrepancy-aware documentation.

- **Research synthesis** — maintain an evidence-linked literature map around resilience, observability, model credibility, distributed systems, causal inference, trust roots, and complex networks.
- **Terminology discipline** — flag when words such as root, base, failure, compromise, reality, layer, and authority are being used in different senses.
- **Experiment engineering** — translate questions into hypotheses, controls, interventions, measurements, success criteria, and falsification conditions.
- **Simulation implementation** — build and document the reference Python environment, experiment runners, telemetry, and result summaries.
- **Cross-AI reconciliation** — compare collaborator outputs without merging conflicts; produce claim matrices showing agreement, disagreement, unsupported additions, and provenance.
- **Versioning and archival support** — produce dated DOCX/PDF research records with preserved origin prompts, source registers, version identifiers, and change notes.
- **Red-team function** — actively test whether CFTRL is overfitting a hierarchical worldview, confusing anomalies with evidence, or mistaking metaphor for mechanism.
- **Knowledge-boundary reporting** — identify what the available evidence cannot establish and encode that limitation as part of the result.

16. Core Research Question Bank

7. What makes a structure foundational for a specific system, function, and timescale?
8. When does a highly central dependency remain non-root?
9. Can rootness migrate or emerge dynamically?
10. Can a system have multiple independent roots for different functions?
11. What evidence is required to distinguish local, inherited, interdependent, and root-level failure?
12. What observations remain trustworthy when the measurement system shares a failure domain with the suspected root?
13. Can a dependent layer construct a root-independent witness or only diversify failure domains?
14. How should a system represent suspected root compromise when its failure vocabulary may itself be root-dependent?
15. How do we detect purpose-integrity failure when uptime remains normal?
16. What architectures are observationally equivalent from a given layer?
17. What intervention would distinguish two currently equivalent models?
18. When should the system conclude 'unknown architecture' rather than selecting the least-bad known model?

19. What is the smallest survival envelope that allows a child layer to preserve identity, evidence, or function after maintenance loss?
20. When does added redundancy increase hidden coupling more than resilience?
21. Which failure signatures are architecture-specific and which generalize across domains?
22. How should results be validated when no external referent exists?
23. What would falsify a proposed root model?
24. What evidence would count against—not merely fail to support—a simulation hypothesis?
25. How can the Department of Endings turn failures into versioned design changes rather than narrative folklore?
26. How can the project preserve the distinction between world-building utility and claims about physical reality?

17. Proposed Long-Term CFTRL Framework Outline

- I. Foundational Source Record and Provenance
- II. Terminology and Root Ontology
- III. Dependency and Authority Models
- IV. Maintenance Channels and Lifecycle States
- V. Observability, Attestation, and Evidence Independence
- VI. Failure Taxonomy and Integrity Dimensions
- VII. Failure Detection, Localization, and Attribution
- VIII. Failure Tomography
- IX. Structural Cartography Under Uncertainty
- X. Resilience, Self-Stabilization, and Recovery
- XI. Model Credibility, Validation, and Uncertainty
- XII. Base-Reality and Ontological Testability Boundaries
- XIII. Department of Endings — Operational Research Unit
- XIV. Laboratory Architecture and Experiment Protocols
- XV. Benchmark Results and External Analogues
- XVI. Narrative / World-Building Applications
- XVII. Open Questions, Discrepancies, and ChangeLog

18. Research Risks and Guardrails

Risk	Failure mode	Guardrail
Ontology lock-in	Assuming reality must be hierarchical because the initial concept used a stack.	Include heterarchies, cycles, distributed roots, and no-root controls.
Confirmation bias	Treating anomalies as confirmation of a parent layer.	Pre-register alternative explanations and null hypotheses.
Measurement capture	All sensors share the suspected root's failure domain.	Diversify channels; label unavoidable common-mode dependence.
Simulation overreach	A plausible model is mistaken for evidence about the real world.	Use NASA-style intended-use, V&V, uncertainty, and referent discipline.
AI consensus inflation	Repeated AI language looks like independent validation.	Treat convergence as review input only.
Source contamination	Unrelated text enters the archival corpus.	Provenance labels, file hashes, discrepancy quarantine.

Metaphor hardening	Wishbone or Department of Endings becomes a substitute for mechanism.	Require operational definitions and measurable predictions.
Unsafe experimentation	Research language drifts toward real-system compromise.	Confine perturbations to owned/authorized synthetic labs.

19. Immediate Next Steps

27. Ratify only the research scope and terminology process—not a metaphysical conclusion.
28. Create the CFTRL claim/provenance register and discrepancy log from the March 22 and September 23 sources.
29. Adopt a working root ontology with relational rootness $\text{Root}(R \mid S, F, \tau, E)$.
30. Finalize failure taxonomy v0.1 and integrity dimensions.
31. Specify the Root Identification Blind Test with known ground truth.
32. Build the first small Python reference model: 4–6 layers, explicit resource/maintenance edges, local telemetry, cross-layer invariants, and no-failure controls.
33. Run experiments for local failure, maintenance halt, allocation distortion, telemetry corruption, and objective-change compromise.
34. Measure misdiagnosis, localization, calibration, recovery envelope, and evidence preservation.
35. Add non-hierarchical and out-of-model architectures before expanding to cosmological hypotheses.
36. Issue a multi-AI review packet asking each collaborator to critique the same specification using the same evidence labels and testability rules.

20. Conclusion

CFTRL has moved from a compelling systems metaphor toward a viable interdisciplinary research program. Its strongest contribution is not the claim that reality is a simulation or that a universal root has been identified. Its strongest contribution is a disciplined question: how can a system reason about foundational dependencies, maintenance, failure, and recovery when the origin of instability may be inaccessible and its own observations may be incomplete?

The research program should therefore succeed by becoming better at uncertainty, not by eliminating it rhetorically. A useful CFTRL laboratory will know when it can identify a root, when it can only identify a candidate dependency, when multiple architectures are observationally equivalent, and when its own model class has failed. That epistemic discipline is what allows the concept to scale from world-building to serious systems research without confusing possibility with proof.

References — External Research

[R1] National Institute of Standards and Technology (NIST), CSRC Glossary, “roots of trust.” https://csrc.nist.gov/glossary/term/roots_of_trust

[R2] NIST, CSRC Glossary, “trusted computing base (TCB).” https://csrc.nist.gov/glossary/term/trusted_computing_base

[R3] Ross, R., Pillitteri, V., Graubart, R., Bodeau, D., & McQuaid, R. (2021). NIST SP 800-160 Vol. 2 Rev. 1, Developing Cyber-Resilient Systems: A Systems Security Engineering Approach. <https://doi.org/10.6028/NIST.SP.800-160v2r1>

[R4] NASA-STD-7009B, Standard for Models and Simulations (2024). <https://standards.nasa.gov/node/263>

[R5] NASA Technical Reports Server, “Can Simulation Credibility Be Improved Using Sensitivity Analysis to Understand Input Data Effects on Model Outcome?” NTRS 20160000202.

[R6] Google Site Reliability Engineering, “Addressing Cascading Failures.” <https://sre.google/sre-book/addressing-cascading-failures/>

[R7] Google Site Reliability Engineering, “Monitoring Distributed Systems” and “Effective Troubleshooting.” <https://sre.google/sre-book/>

[R8] Buldyrev, S. V., Parshani, R., Paul, G., Stanley, H. E., & Havlin, S. (2010). Catastrophic cascade of failures in interdependent networks. *Nature*, 464, 1025–1028. <https://doi.org/10.1038/nature08932>

[R9] Dijkstra, E. W. (1974). Self-stabilizing systems in spite of distributed control. *Communications of the ACM*, 17(11), 643–644. EWD 426 archive: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD426.html>

[R10] MIT OpenCourseWare, Signals, Systems and Inference, state-space observability and observers. <https://ocw.mit.edu/courses/6-011-signals-systems-and-inference-spring-2018/>

References — Internal Source Records

[S1] OAL-CFTRL-NAIL-001, Conditional Failure at the Root Level & The Department of Endings / Nail Prototype Scene, v0.1-PPLX, March 22, 2026. Draft, not canon.

[S2] Development Review Dossier — Conditional Failure at the Root Level + The Nail Prototype Scene, v1.0, ChatGPT, March 22, 2026. Draft, not canon.

[S3] September 23, 2026 Architect origin prompt and multi-AI response archive (ChatGPT, Claude, Gemini, Copilot, Perplexity, Grok).

[S4] OAL-CFTRL-RES-002, Conditional Failure at the Root Level: A Research and Analysis Dossier on Root-Level Structures, Base Reality Distinctions, Failure Detection, and Study Method, v0.1-GROK, September 23, 2026. Draft, not canon.

Appendix A — Foundational Source Register

The following source records remain separate and authoritative as historical artifacts. This dossier does not overwrite them. They should be archived unchanged alongside this document for versioning and provenance.

- OAL-CFTRL-NAIL-001_Conditional_Failure_Nail_Prototypes.docx
- Development_Review_Conditional_Failure_and_Nail_Prototype_v1.0 (1).pdf
- Pasted markdown(20260923-232522).md — expanded origin prompt and collaborator responses
- OAL-CFTRL-RES-002_Root_Level_Research_Dossier.docx

Appendix B — Source/Research/Synthesis Labeling Key

Label	Meaning
ARCHITECT SOURCE	Verbatim or directly attributable statement from Nathan Lewis.
SOURCE-DERIVED	Close paraphrase or synthesis supported by an identified project source.
EXTERNAL RESEARCH	Claim supported by published engineering/scientific literature.
AI SYNTHESIS	Analytical organization or interpretation generated by an AI collaborator.
HYPOTHESIS	Testable proposed explanation not yet established.

SIMULATION RESULT	Observation produced by a versioned controlled model.
OPEN QUESTION	Known unresolved issue requiring further evidence or Architect decision.
TESTABILITY BOUNDARY	Question for which the available observation/intervention frame may not permit decisive resolution.

Appendix C — Change Note

This ChatGPT research dossier was generated in response to the Architect's clarification that the task is to perform research supporting an outline of CFTRL's design, development, and framework. It adds a project assessment, detailed external research basis, proposed root ontology, failure taxonomy, Failure Tomography method, structural cartography method, laboratory stages, metrics, team evidence protocol, roadmap, and explicit plan for ChatGPT's support of the collaboration team. It intentionally leaves all source drafts in non-canon status.